VA55 - TP1 rendu

PERRIN Antoine, BEAUJARD Traïan

Ce qu’on cherche à faire

On veut que le robot suive la ligne sans passer son temps à zigzaguer. Le capteur est à gauche : on lui fait suivre le bord gauche de la ligne.

On prend 50 comme valeur à viser, entre le noir (0) et le blanc (100). Pour voir si on s'en éloigne, on trace l'erreur :

$$e = x - 50$$

Quand le capteur voit trop de blanc, on corrige vers la droite. Quand il voit trop de noir, on corrige vers la gauche.

Le réglage utilisé sur le tapis

En se basant sur les différents termes P, I, D, expliqués ci-dessous, nous avons paramétré nos 3 gains.

Le terme P donne la correction principale : plus on s'éloigne de 50, plus il corrige.

Le terme I garde une correction quand l'erreur dure. C'est utile dans un virage, mais il peut aussi continuer à pousser alors qu'on revient déjà vers le bord.

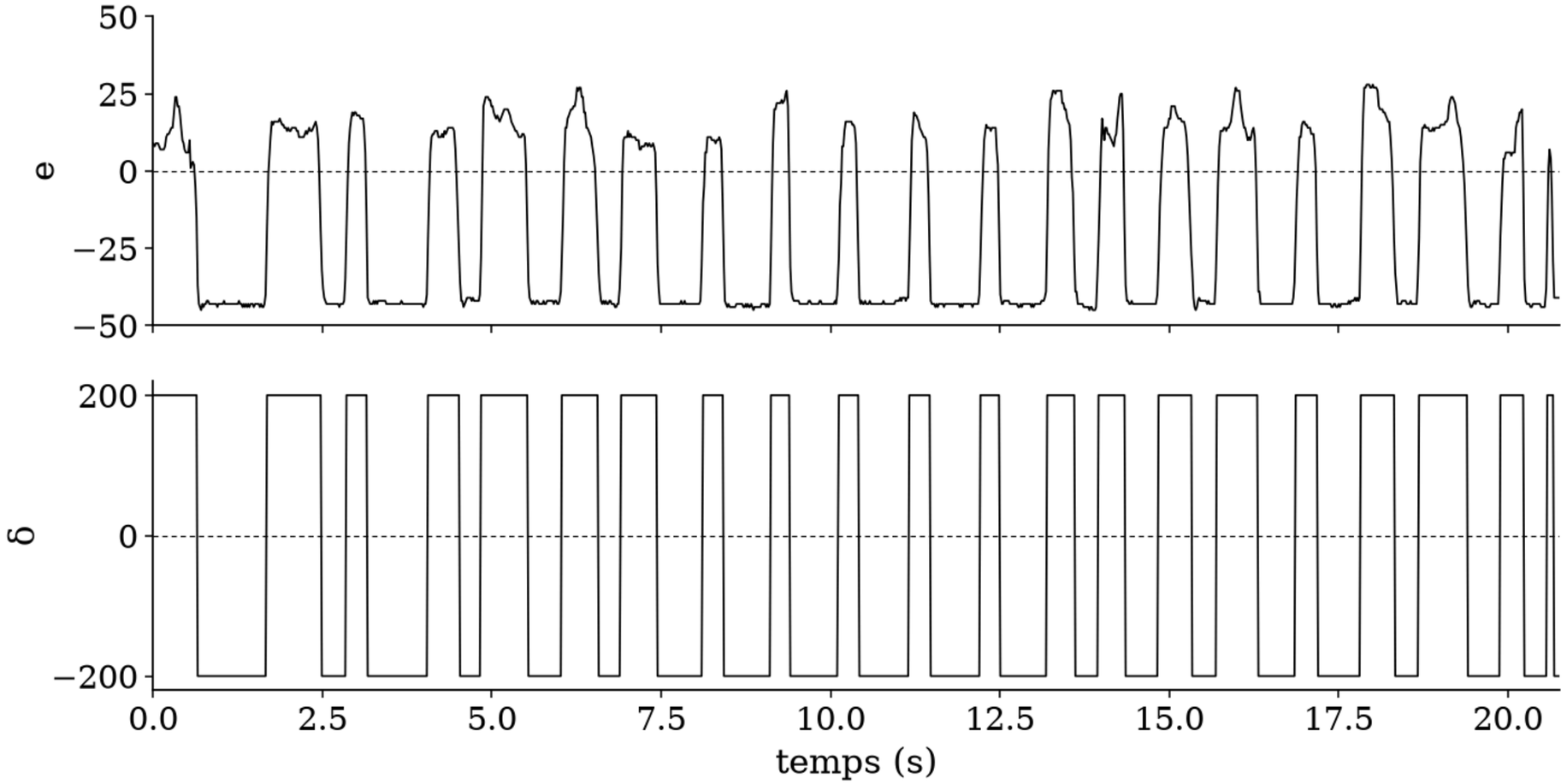
Le terme D réagit aux changements rapides de l'erreur. On le filtre pour éviter que chaque petite variation du capteur fasse changer brutalement la commande.

kp = 2 ; ki = 1,70 ; kd = 0,035.

Bang-bang (ligne droite)

20,8 s

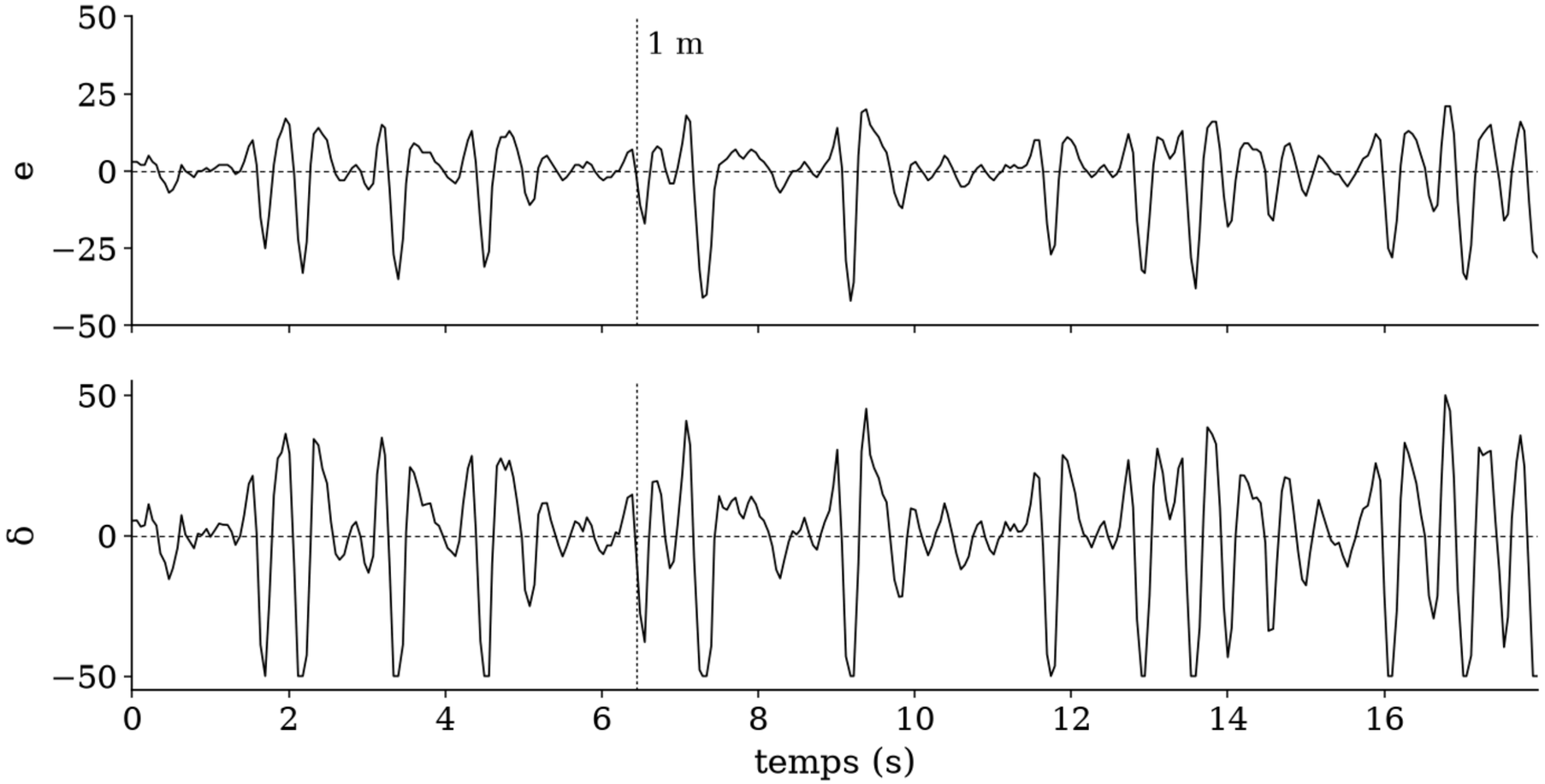
La commande passe directement de +200 à -200 : on voit bien les allers-retours sur la courbe. Autre détail : elle bascule à une réflexion de 20, alors qu'on trace l'erreur par rapport à 50. On ne vise donc pas le même point qu'avec le PID.



Ligne droite avec kp = 2 ; ki = 1,70 ; kd = 0,035

18,0 s

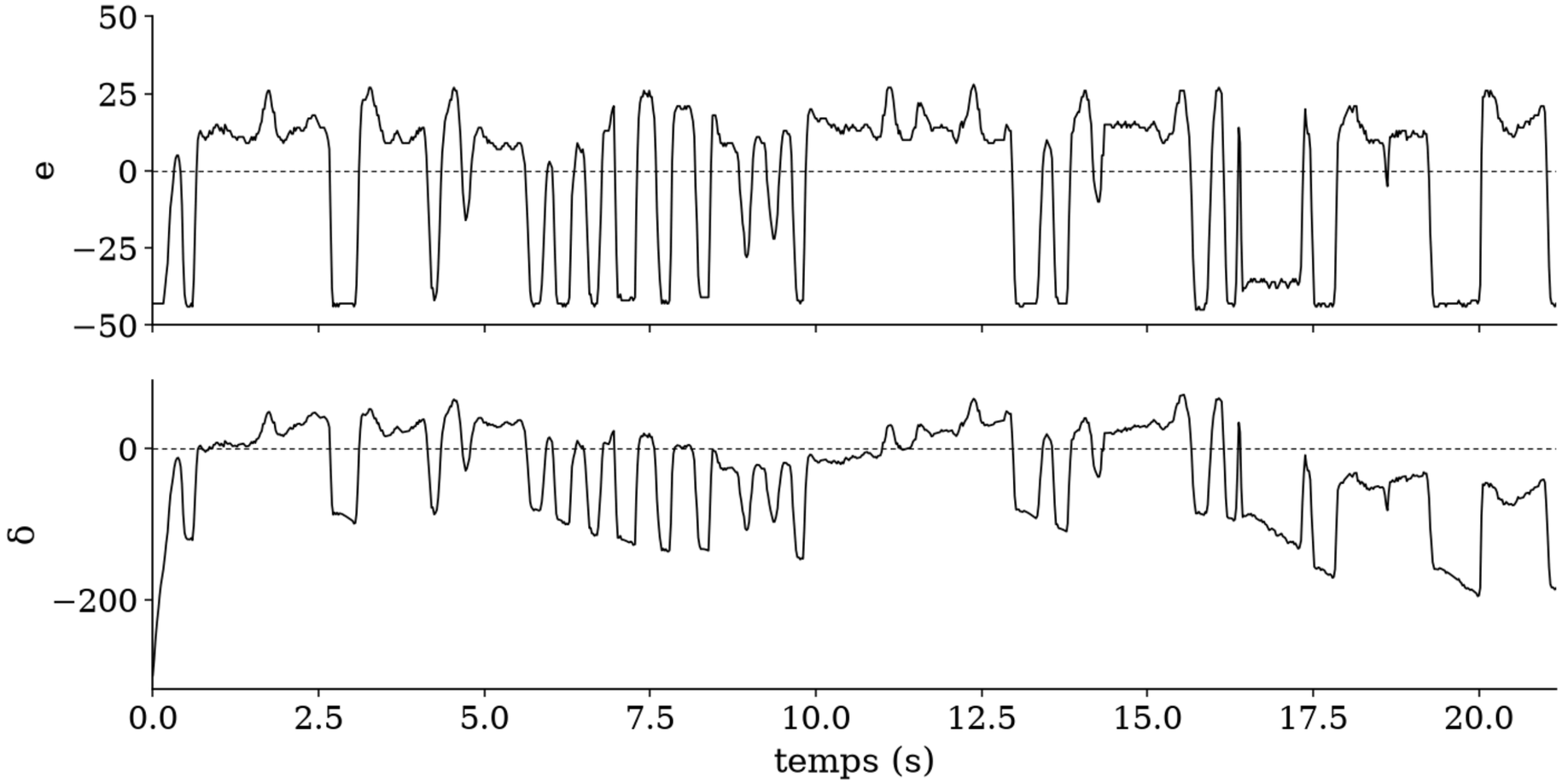
Sur cette droite, l’erreur reste souvent près de zéro, avec encore des écarts brusques. Nous avions un repère physique à 1 m. Le trait à 6,45 s correspond au temps estimé pour l’atteindre à 155 mm/s ; les 18 s couvrent tout l’enregistrement.



Tapis de course kp = 2 ; ki = 1,70 ; kd = 0,035

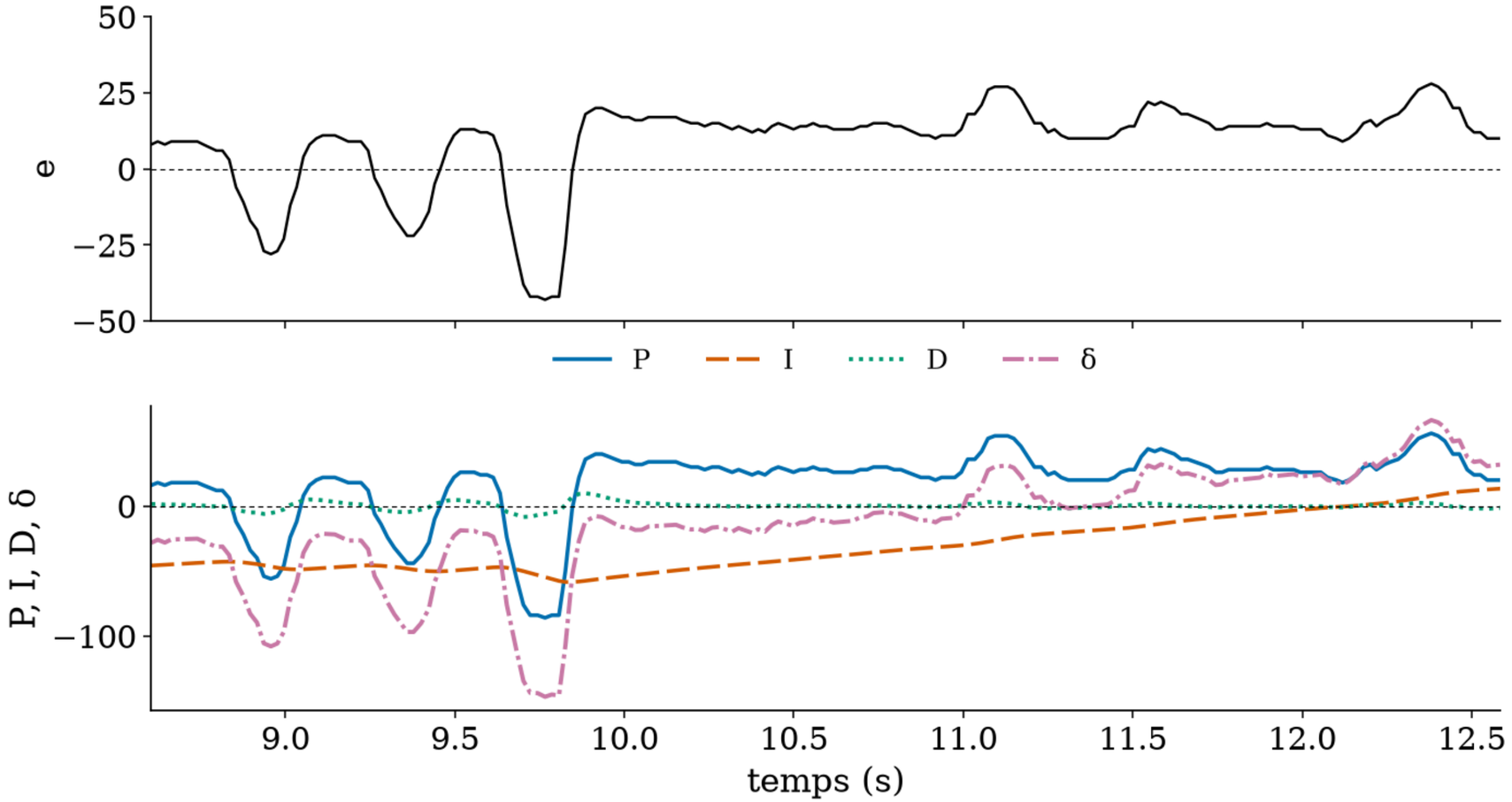
21,2 s

Le parcours du tapis dure environ 21,2 s, avec un arrêt manuel. L'erreur revient plusieurs fois vers zéro, mais elle reste aussi un moment près de -44 : le capteur voit alors surtout du noir. Le suivi fonctionne sur ce parcours, mais il reste irrégulier.



Un seul essai, les trois contributions du PID

Zoom de 8,6 à 12,6 s sur le même log du tapis. En haut : l'erreur. En bas : les contributions calculées par chaque terme du PID; P en bleu, I en orange, D en vert, à chaque instant, P, I et D s'additionnent pour donner la commande δ en rose.



Ce qui a amélioré la stabilité

Pendant nos essais, nous avons résolu le problème de stabilité en faisant calculer la correction plus souvent : nous avons réduit l'attente à period_ms ~= 5 *(temps d'écritures pour légèrement augmenter la frequence)*.
(nous avons perdu les logs de ces meilleurs essais, mais on l'a bien montré lors de la dernière scéance que c'était stable..)

Cela permet de réagir plus tôt quand le capteur s'éloigne du bord, au lieu de laisser le robot continuer trop longtemps avant de corriger.

Code de la commande, extraits, calcul des termes

```
# tpa/RobotLineFollower.py
# Gains PID : 2 / 1.70 / 0.035
# init avant boucle :
integral = 0
previous_error = 0
previous_time = watch.time()
filtered_derivative = 0

# boucle PID :
reflection = self.robot.sensor.getReflection()
error = reflection - self.setpoint
current_time = watch.time()
dt = max(current_time - previous_time, 1) / 1000.0
raw_derivative = (error - previous_error) / dt
filtered_derivative = (
    self.derivative_filter * raw_derivative
    + (1 - self.derivative_filter) * filtered_derivative
)
derivative = filtered_derivative
previous_error = error
previous_time = current_time
integral_candidate = integral + error * dt
integral_candidate = self._limit(
    integral_candidate, -self.integral_limit, self.integral_limit
)
```

Code de la commande, extraits, correction des roues

```
# suite boucle pid :
proportional_derivative = self.kp * error + self.kd * derivative
candidate_steering = (
    proportional_derivative + self.ki * integral_candidate
)
if not (
    abs(candidate_steering) > self.max_steering
    and candidate_steering * error > 0
):
    integral = integral_candidate
steering = proportional_derivative + self.ki * integral
steering = self._limit(steering, -self.max_steering, self.max_steering)
self.robot.drive.drive(self.speed, steering)

# RobotDrive.drive : vitesses moteur, steering_gain = 2.0
correction = self.steering_gain * steering
left_speed = speed + correction
right_speed = speed - correction
self.left.run(-left_speed)
self.right.run(right_speed)

# Fonction de saturation, RobotLineFollower._limit :
return max(minimum, min(value, maximum))
```

Ce qu’on retient des logs

Pour comparer l'amplitude des écarts, on prend leur valeur absolue puis on fait la moyenne. Cela évite qu'une erreur à gauche annule une erreur à droite.

Mesure	Bang-bang	Droite PID	Tapis PID
Durée de l'enregistrement	20,8 s	18,0 s	21,2 s
Écart moyen en valeur absolue	29,8	7,9	20,7
Plus grand écart en valeur absolue	45	42	45

Ces écarts sont des points de réflexion du capteur, pas des millimètres.

La droite est l'essai où l'erreur est la plus faible. Sur le tapis, elle reste beaucoup plus grande : le suivi est moins régulier.

Les conditions et les réglages sont différents. On peut décrire ces résultats, mais pas attribuer toute la différence au PID.

Bilan du TP

Les logs montrent la différence entre une correction qui bascule d'un côté à l'autre et une correction qui dépend de l'erreur et de son évolution.

Sur les logs conservés, la droite reste plus près de la valeur visée que le tapis. Ensuite, réduire l’attente à period_ms = 5 a résolu notre problème de stabilité pendant les essais ; nous n’avons pas sauvegardé les logs correspondants.

Nous avions un repère physique à 1 m sur la droite. À la vitesse de consigne de 155 mm/s, cela donne un temps estimé de 6,45 s. Les essais ont été arrêtés manuellement ; nous n’avons pas de chronométrage séparé au passage du repère.

La droite a aussi été enregistrée avec d'autres gains que le tapis : 1,7 / 0,9 (1,1 côté blanc) / 0,06, contre 2 / 1,70 / 0,035. Nous n'avons donc pas les deux essais avec le réglage définitif demandé.

Nous n'avons pas sauvegardé nos meilleurs essais malheureusement. Nous présentons ici ceux dont nous avons gardé les données. Les meilleurs essais, battant les autres d'au moins 10 à 20%, étant des temps sur ligne droite et le 8 en 16-16.5 secondes.